

Video Transcript

Video 4: Project 1: Creating a Books Web Application Solution (08:02)

So, our first task was that we needed to be able to display the books with the images. So, let's put in test user and we see that we get welcome test user. And if we hit the books, now we see that we get a nice print out; and here we've got the images being rendered and the details of the book. And we need to be able to then add a number of users. So, I added myself and I gave myself just reader permission. Now, you're going to be an admin, but a reader, I can't add a book, admins only. Similarly, I can't add an image, but I can still display the books. So, that works for John. Now, let's see an admin. I've got an admin called Anne and Welcome Anne. Add a book. Anne can add a book. Well, let's put something decent. Anne Harding, title, Lost Worlds.

So, we can submit that and we'll see that we get a book added. Let me scroll up a little bit just so you can see it. There you go. So, we have that and I've got some added images that I loaded in. So, it gets an image automatically there, but I can add an image that I like. It's the fourth book. So, I put 4 there and let me select an image. So, let's have a look here. Well, I've put something that I know; I think this is me. So, we upload the image and then we can go back. So, I'll go back now and hit 'Books', and it looks like I overwrote the wrong one. We need to go and fix that. So, what I'm doing is I'm deleting the cache. So, I believe that it was rendering the wrong image because of caching. Let's see a missing cookie. I deleted my cookies, so I need to go back and log in. So, let's go back to 5000. Let's pretend that I'm Anne. Okay, submit. Let's take a look at the books. I put my picture there and now, it comes, it gets rendered.

So, be careful of caching that. You need sometimes to delete everything, otherwise you're getting a cached image of the page, but that's all working. So, that was all that we needed to do. We've got our roots protected. So, remember, I couldn't hit Add Book or Add Image, whereas Anne can and you should be able to as well because you're going to give yourself admin permission. Let's quickly look at the code that was necessary to do this. I'll put the solution up on GitHub so you can download it. But looking at the code here, you see, we had some images for the books and then we uploaded some other images. In books, the trick there was we needed to put an image tag and the source. Then we needed to make sure that we loaded the right image.

So, I decided to call the images by name, image, and I put the `{book [ 'id' ]}}`, so 0, 1 or 2; so that'd be image 1, etc, 2, and I put the width as 50. You can put it as whatever you like. So, that was to render the books. Then, we needed to go and add in users. So, that was fairly straightforward. We just put the users in. So, what we needed to do is we needed to make sure that the check on validUsers, so checkUser. Here, we needed to make sure that that worked. And then, I returned the user's name and their role, but not the password. We don't want to put the password in the token, that would be dangerous.

Then, what we do is, we put into the access token these added user_claims that I'm setting to this dictionary "role", and I'm putting into that the validUser's role. So, that gets embedded into the JSON web token. We get the response object, so we had to add make_response. And then we're adding into the response the status_code, and also, we're getting the cookies. So, we set the access cookies in the response object and we put the access token there and then we 'return response'. So, that's going to get sent back to the user. The user's going to store it in their cookies and they're going to return it to us whenever they return. Now, what we needed to do was to protect the routes. So, here we had JSON Web Token required, jwt_required, but on addbook and addimage, we wanted to use this decorator admin_required. Let's go and look at that. You needed to understand the structure of the decorators. The decorator is itself using a decorator. So, to use @wraps and this function, you had to load up. Here, you needed to import wraps from functools and then you needed to verify the JWT that it was in the request and then get out of that the claims.

And then, if the claims, and we know that we put role into the claims, if that was not equal to admin, then we return an error message, 'Admins only'. Otherwise, we return the function that's passed in with its arguments. So, that'll be whatever the function is below the decorator. So, that was it. So, download it from GitHub and run it for yourself. I'm sure there are other slightly different solutions, but as long as it works that would be fine. Okay, bye for now.